ECE 350: Real-Time Operating Systems Winter 2027

Lecture 31 — Live Kernel Patching

Jeff Zarnett *2026-08-15*

Can't Stop, Won't Stop

When people are counting on a system – particularly life and safety-critical ones – it might be difficult to schedule a maintenance window for it. Let's say that we cannot, or at least do not want to, have a system restart for the purposes of updating the kernel. It is perhaps not the right choice to disrupt a long-running task, but also not great to leave security vulnerabilities unaddressed until a maintenance window that's far in the future. We need a third option: change the kernel without a reboot. Or in other words, we are trying to maximize availability.

Many modern software projects allow for some sort of staged or transitional rollout where you start sending new requests to the new system and let the old system finish out old requests before you shut it down. But the kernel of the operating system doesn't permit this sort of thing; there can be only one kernel and we can't start a second one and shift things over to the new one. Not on the same machine anyway – we could do this kind of warm handover if we're replacing a machine with another. But that's not what we want to talk about here, we're thinking about updating the currently-running kernel.

As we learned earlier, kernels have the ability to load and unload new modules. So if what we need is to unload and reload a kernel module, that's not too difficult. Let's say the vulnerability we want to fix is that there's a bug in the device driver for an attached sensor. Great, we can probably pause (quickly!) things that use it, unload the kernel module, load the new one, and resume execution. Seems possible given what we know.

But if the bug is in the kernel itself? That requires changing the kernel's main functionality. And doing it live. Can we load some code into the kernel and have it replace existing things?

In practice, this is a little bit like the kernel doing a surgery on itself. There is a precedent of sorts in the real world: a Soviet doctor who was stuck in Novolazarevskaya Station, Antarctica needed to do a surgery to remove his own appendix[1]. That was perhaps a little risky, but what choice did he have? It was 1961 in Antarctica. Still, it's doable if we adhere to some restrictions and do it well.

The basic plan has a few steps. Step one is validating that we can even make this change at all; changes to functions are fine, but data structures cannot be changed on the fly. Then, prepare the patch; making the compiled binary code that we intend to replace some part of the existing kernel. Then we need to make sure that it's safe to make the swap (details below in the implementation description) and execute the replacement. After that, future system calls then run the new code. This high-level view makes it seem easy, but the challenge is in the details, particularly the part about making sure it's safe to make the swap.

The good news is that many patches are relatively straightforward in terms of the size of the change. In fact, the Kernel docs around this say: "Many fixes do not change the semantic of the modified functions. For example, they add a `NULL` pointer or a boundary check, fix a race by adding a missing memory barrier, or add some locking around a critical section. Most of these changes are self contained and the function presents itself the same way to the rest of the system."[2] In other words, we're changing a few lines within the function, but the function's main logic, purpose, signature, return type, etc. do not change.

This makes the size of the problem a lot more manageable than wholesale replacement. Though, in theory, the patching mechanism can be used to do a larger change than just one of these "typical" cases. The mechanism does not have size restrictions, though for a large change in terms of lines of code or changing the signature of

[1] `https://en.wikipedia.org/wiki/Leonid_Rogozov`
[2] `https://docs.kernel.org/livepatch/livepatch.html`

something, it would be right for a code reviewer to question whether the change is minimal or if this ie even the best approach to solving the issue. (Ask me some time about using the Ruby monkey-patching mechanism on Rails `ActiveRecord` to parameter-bind large arrays of IDs in SQL `IN(...)` queries.)

As mentioned when learning about kernel modules earlier, the consequences of getting this wrong are potentially severe. It could lead to a crash (kernel panic), but it could also lead to silent corruption of the system and its data. Or we could *introduce* some security vulnerabilities or data leakage, which is rather the opposite of the purpose.

Patching with `kpatch` and `kGraft`

If you're familiar with Linux and open source software in general, it won't surprise you to know that there were two competing approaches to this that came out around the same time in 2014: `kpatch` and `kGraft`. The final version of live patching in the kernel took some parts of each of these, but we can examine each of them individually and then see how they came together in the end.

A quick walkthrough of the challenges and differences between these approaches will help. Fortunately, one exists [Cor14]. Both mechanisms aim to do a replacement of a function within the running kernel; the new code is loaded as a module (akin to a driver) into that kernel. Then, calls to the old function are intercepted and redirected to the new implementation.

To prepare the patch, write the new version of the code and compile it. Put the changed function(s) into a kernel module to prepare it for loading.

There's a little nuance about how to find the place to patch in the compiled kernel. At first glance, you might think that the approach here is to just search for the right function in the compiled binary, and edit the instructions of the kernel as it exists in memory (just overwrite these 1s and 0s with the better ones!). This does not work. Even if we do not have to worry about the code sections being different in size – and they almost certainly are if we are changing the logic or adding something, we forgot something important. Replacing function `f()` *while* it is executing does not work; it can be replaced if nobody is executing it right now. So, is anybody executing it right now? That has to be accounted for.

In any event, recognizing that we want the ability for a patch to be a different size than the original binary code (without resorting to any weird trickery where we rewrite more than just the function we want to change), there's a better way. We can redirect execution instead of overwriting the old version.

That's the job of `ftrace`, the kernel tracing infrastructure[3]. The original purpose of this was, as the name suggests, to be able to record the execution flow of the kernel. This is a debugging and programming-for-performance technique that lets us both understand what functions call what other functions, but also how long things take. The `ftrace` implementation allows the user to attach callbacks to a function. For performance or debugging purposes, you would want to be taking notes about the execution state.

Normally, those callbacks would be observation-based and not change the function being observed. But the key insight here is that `ftrace` can register ANY hook you like, including one that redirects control flow. That does not break any rule, because there are few restrictions on what can be in the callback. The `ftrace` docs do talk about restrictions on them about the fact that they can get called from anywhere and the risk of recursion resulting in an infinite loop or stack overflow. But nothing prevents the redirection of control flow. So it's legal, even if it might not have been intended by the people who created `ftrace`.

To apply the change, use `ftrace` to add a hook that is triggered before running function `f()` so that it sends control flow over to `f2()` and then `f2()` executes. The return from `f2()` answers the original call (returns to where `f()` would have returned to). Can we do that? Sure, remember from the execution model conversations that the keyword `return` in C looks for a return address further up the call stack so we know where to pick up in the calling function. We do not have to go back to the hook, and the fact that we never entered `f()` does not matter. Control of execution flow is pretty much the kernel's favourite thing to do, so this is easy.

[3] `https://docs.kernel.org/trace/ftrace-uses.html`

The kGraft approach keeps the old and new versions of f() around and each process that's currently executing with the old version can continue, but new calls go to the new function. This is similar to what was referenced earlier in the application rollout view of the world: let old requests finish in the old system, then pull the plug on the old one when all old calls are done.

The kpatch approach is a little more disruptive and feels more like surgery or Java's (older) garbage collection mechanism: it stops all program execution on all CPUs (briefly), then looks at the execution stack of every running process to see if any of them are inside the function being patched. If yes, then the patch fails and we'll have to try again later; if no, the patch can be applied – everyone sees the new version from then on – and then we can resume execution of other processes.

It's acknowledged in that article that there are some functions that can never be patched in the kpatch approach because they will *always* be running somewhere inside the kernel, including the scheduler. One may hope this is a minority of situations.

Livepatch

As previously mentioned, the final version that went into the kernel takes some from each of the two proposed approaches above and creates what its own docs describe as a hybrid approach. Let's go off the doc to understand it[4]; this subsection walks us through those docs.

The term *consistency model* is used to describe the behaviour of the live patching functionality. The model is used to have some assurance that the system's behaviour is consistent.

The docs introduce a new scenario that we did not previously consider: if the patch changes lock ordering in multiple functions, we actually want those changes to take effect all at once instead of one at a time (otherwise, we could get a deadlock from lock ordering problems!).

The approach of the live patch system is the hybrid of the kpatch and kGraft approaches. It takes the kGraft approach of doing the transition over time on a per-task basis; but it uses the kpatch approach for stack trace switching. When it's safe to switch over, a task moves from the old version to the new version of the function. Over time, all tasks eventually make it to the new version.

Ah, but when is it safe to switch over? The docs say there are three possibilities:

1. Stack checking: like the kpatch approach, it looks at the stacks of all the tasks not currently running. If it is not running the function(s) being patched, no problem, they are switched over immediately. It is likely that most tasks are patched on the first try, but it's possible to retry as much as necessary.
2. Kernel exit task switching: notice when a task exits from the kernel (syscall ends, signal handler, etc) and patch it at that time. This works even for a task that's in an infinite loop that doesn't make system calls; it will *eventually* get interrupted, even if it's just by the scheduler timer.
3. Idle "swapper" tasks are a part of the OS and don't exit the kernel, so they have their own check in their loop to figure out when to make the switch.

Only one patch can be in the process of being applied at a given time and there is no time limit for how long it can take to apply it. There are the following states for a live patches life cycle: loading, enabling, disabling, removing, and a bonus one, replacing. Replacing and disabling are mutually exclusive.

Loading. Loading sounds like it's what happens when you bring the module into the kernel, and it is, but it triggers the enabling step immediately in its initialization. It happens this way for some internal reasons, but primarily in case an error in enabling happens, which results in rejecting the load of the module.

[4] https://docs.kernel.org/livepatch/livepatch.html

Enabling. Enabling begins application of the patch and the new version starts to get used. Running tasks are moved over to the new version according to the consistency model rules until they're all moved.

Disabling. Disabling starts the reverse of the process of enabling, just as the name would lead you to believe. The previous version starts to get used and tasks move from the new version to the old one until they're all migrated. The ability to reverse a patch lowers the risk, to some degree, since we can always validate and see what happens.

Removing. If the patch has been disabled and no uses of it exist anymore, it can be removed.

Replacing. As if live-patching isn't complicated enough, there exists an even more complex version of this: Atomic Replace and Cumulative Patches, which is complicated enough to get its own documentation page rather than being a section of the live patching doc[5].

Replacing is used in scenarios when there are dependencies between different patches. The goal is to group up several things into a cumulative patch and make it so we can do an atomic replacement. This does clean up the list of applied patches, but maybe that's not a good reason to do it because it becomes very hard to unapply a cumulative patch.

Before going down this path, my recommendation is to pause and ask whether this is actually a good idea. Applying some patches to the kernel like this should be rare and for emergencies, and ideally only temporary until it's possible to have the next maintenance window.

Minimal Example Let's look at this minimal example from the kernel source examples[6]

```
// SPDX-License-Identifier: GPL-2.0-or-later
/*
 * livepatch-sample.c - Kernel Live Patching Sample Module
 *
 * Copyright (C) 2014 Seth Jennings <sjenning@redhat.com>
 */

#define pr_fmt(fmt) KBUILD_MODNAME ": " fmt

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/livepatch.h>

/*
 * This (dumb) live patch overrides the function that prints the
 * kernel boot cmdline when /proc/cmdline is read.
 *
 * Example:
 *
 * $ cat /proc/cmdline
 * <your cmdline>
 *
 * $ insmod livepatch-sample.ko
 * $ cat /proc/cmdline
 * this has been live patched
 *
 * $ echo 0 > /sys/kernel/livepatch/livepatch_sample/enabled
 * $ cat /proc/cmdline
 * <your cmdline>
 */

#include <linux/seq_file.h>
static int livepatch_cmdline_proc_show(struct seq_file *m, void *v)
{
        seq_printf(m, "%s\n", "this has been live patched");
        return 0;
}
```

[5] https://docs.kernel.org/livepatch/cumulative-patches.html
[6] https://github.com/torvalds/linux/blob/master/samples/livepatch/livepatch-sample.c

```
static struct klp_func funcs[] = {
        {
                .old_name = "cmdline_proc_show",
                .new_func = livepatch_cmdline_proc_show,
        }, { }
};

static struct klp_object objs[] = {
        {
                /* name being NULL means vmlinux */
                .funcs = funcs,
        }, { }
};

static struct klp_patch patch = {
        .mod = THIS_MODULE,
        .objs = objs,
};

static int livepatch_init(void)
{
        return klp_enable_patch(&patch);
}

static void livepatch_exit(void)
{
}

module_init(livepatch_init);
module_exit(livepatch_exit);
MODULE_DESCRIPTION("Kernel Live Patching Sample Module");
MODULE_LICENSE("GPL");
MODULE_INFO(livepatch, "Y");
```

Looking at this briefly, we have, in order.

1. The new function written that is meant to replace an existing one
2. The mapping that says what the old and new functions are
3. The mapping that says what module we are applying the patch to (here, it's the core of the kernel)
4. Registering the module as the patch owner
5. Initialization that kicks off the application of the patch
6. The removal. Does nothing here; there's nothing to clean up except sending execution back to the old name
7. The module configuration stuff at the bottom that the kernel needs to recognize and accept a module

Shadow Variables

The live patching implementation also allows something that the `kpatch` approach did not: changing kernel data structures! Well. Sort of changing them. It's more like, what if we gave them companions. Let's go through the documentation[7].

Shadow variables allow attaching these extra companion objects to the existing data structures. So we're not really changing the originals, but we do get to put more things in the associated object. It's permitted to have multiple shadow variables attached to the same parent.

[7] `https://docs.kernel.org/livepatch/shadow-vars.html`

This implies a need to link these things. A shadow object has some identifier that tells us what the parent is; the kernel must also have a hash table to match.

This is still C, after all, so there's a need to allocate the shadow variable structure and deallocate them explicitly. The patch code will create the shadow objects with a function call to do so; to accommodate the existing structures before the patch was applied, a lazy load-or-create approach is needed. To get rid of the shadow object, that's also something to do explicitly when the parent is deallocated, but getting rid of all created shadow objects is also necessary when unloading the patch if that happens.

References

[Cor14] Jonathan Corbet. The first kpatch submission, 2014. Online; accessed 10-July-2026. URL: `https://lwn.net/Articles/597407/`.